EdgeTPU Firmware Recovery and Fault-Oracle Analysis Artifacts

A Artifact Appendix

A.1 Abstract

The artifact provides a three-stage EdgeTPU firmware-analysis workflow: acquiring memory dumps on a prepared Pixel 9a, importing dumps into IDA and recovering candidate functions, and scanning reconstructed firmware for candidate fault-oracle memory-access gadgets. The package includes acquisition source and binaries, two reference logs, a firmware image, IDAPython scripts, and a prepared IDA Pro 9.4 database. Reviewers can start from either the device, supplied logs, or prepared database. The package's redistribution license remains to be specified; IDA and Hex-Rays are proprietary dependencies and are not bundled.

A.2 Description & Requirements

The root `README.md` documents all commands and expected output formats. `POC/DumpAuto_stable/` contains `runner`, its source, the `cracktpu` acquisition helper, and `compile.sh`. `Public_artifacts/` contains the firmware, dump logs, importer, function-recovery script, and scanner. The prepared database is `edgetpu-rio_eval_re.fw.i64` at the package root.

A.2.1 Security, privacy, and ethical concerns. Acquisition requires root, stops the EdgeTPU logging service, and can reboot the device or interrupt SSH. Use the prepared review device or an authorized test device. After reconnecting, re-enter root and stop logging before resuming; restore logging afterward. Import and recovery modify the database, so use a working copy. Scanner reports are candidates, not automatically confirmed vulnerabilities.

A.2.2 How to access. Follow the access workflow in README Section 1.1. The Tailscale invitation and SSH connection details for the prepared Pixel 9a are provided through the artifact-review channel. Join the reviewer network and log in using the supplied credentials. Root access is already enabled: after logging in, run `su` to enter a root shell; no additional rooting steps are required. Contact the authors through the artifact-review channel if the invitation expires or the device is offline. Offline stages need no device connection.

A.2.3 Hardware dependencies. Live acquisition uses the authors' prepared, rooted Pixel 9a with access to `/dev/edgetpu-soc`. Other devices or firmware builds are not assumed compatible. Offline evaluation uses a workstation capable of running IDA Pro 9.4; minimum RAM and disk requirements have not been measured.

A.2.4 Software dependencies. IDA Pro 9.4 with IDAPython is required; Stage 3 also needs a licensed ARM Hex-Rays decompiler. Remote access requires Tailscale and SSH. Rebuilding and local deployment require Android NDK and ADB. The build script uses `darwin-x86_64` and `aarch64-linux-android34-clang`; adjust the NDK host path on other operating systems. SSH access does not imply ADB access.

A.2.5 Benchmarks. Inputs are `Public_artifacts/edgetpu-rio.fw`, the prepared database, and two logs: `0x0ff00000_0x0ff0E4FF_dump.log` (14,656 words; 2,175 tagged) and `0x10000000_0x1000ea80_dump.log` (15,009 words; 2,150 tagged). Records contain an address and a 32-bit value, optionally marked `cache_unstable`. These counts describe bundled inputs; unstable locations need not match byte for byte in a fresh acquisition.

A.3 Set Up

A.3.1 Installation. Extract the package and install the dependencies above. For device acquisition, use the deployed binaries or set `NDK` and run `sh compile.sh` in `POC/DumpAuto_stable/` with an ADB-connected device. After logging in to the prepared Pixel via SSH, run `su` to enter a root shell, then run `stop edgetpu_logging`; enter `/data/local/tmp/`, make both helpers executable, and create `review-dumps/`. For a fresh IDA import, load the raw firmware as ARM little-endian at **base address `0x0fff000`**. Alternatively, open a copy of the prepared `.i64` directly; do not rebase or reimport it.

A.3.2 Basic test. For the shortest offline check, open the prepared database, wait for analysis, and execute `Public_artifacts/faultoracle_scanner_mc.py` via `File > Script file`. The IDA output window should show per-handler reports and end with `TOTAL sites (after per-def-site dedup): ...`. Check for no `function at` or `gen_microcode failed`; a final total alone does not prove that every handler was analyzed. No verified reference site count is supplied in the README.

A.4 Evaluation Workflow

A.4.1 Major claims. The following workflow claims follow the README's result-to-artifact mapping. Exact paper section/table references and quantitative acceptance criteria must be supplied before claiming reproduction of the paper's main results.

- (C1) The acquisition workflow recovers the two memory regions and supports resumption after interruption (E1).
- (C2) Imported dump words support firmware reconstruction and heuristic recovery of candidate function boundaries (E2).
- (C3) The microcode scanner identifies candidate fault-oracle memory-access sites for manual inspection (E3).

A.4.2 Experiments. Measured human effort, runtime, and disk usage for E1–E3 are not provided in the README and remain to be supplied. E1 includes repeated reads and may require reconnection; no runtime bound is claimed.

(E1) **Memory acquisition (C1; README Stage 1).** *Preparation:* Connect, enter root, and stop logging as above. *Execution:* In `/data/local/tmp/`, invoke `./runner START END ./cracktpu OUTPUT` twice: first with `START=0x0ff00000`, `END=0x0ff0e500`; then with `START=0x10000000`, `END=0x1000ea84`. Use the matching log names under `review-dumps/`. Endpoints are exclusive and steps are four bytes. After reboot, reconnect, repeat `su` and `stop edgetpu_logging`, and rerun the same command/output path. Runner resumes after the last valid record. Finish with `start edgetpu_logging`. *Results:* Retrieve logs and check their final word addresses: `0x0ff0e4fc` and `0x1000ea80`, respectively. Inspect instability tags and incomplete acquisitions rather than requiring identical values at every location.

(E2) **Import and recovery (C2; README Stage 2).** *Preparation:* Load the firmware at the stated base and save a copy. *Execution:* Run `import_dump_log_to_ida.py` twice, selecting one supplied log each time; wait for reanalysis. Run `recover_tpu_funcs.py`, checking that `SEG_NAME` (default `ROM`) names the intended segment. Repeat for other relevant segments, such as `.dump_0ff`, as needed. *Results:* Expect 14,656 and 15,009 patched words if all addresses are mapped; investigate malformed/unmapped records. Save recovery output ending in `DONE — added ... funcs total` and inspect candidate functions. Zero additions may reflect prior recovery or a wrong segment.

(E3) **Fault-oracle scanning (C3; README Stage 3).** *Preparation:* Use the prepared database and ARM decompiler. *Execution:* Run the scanner as in the basic test; it analyzes configured `HANDLERS` at `MMAT_CALLS` maturity. *Results:* Save IDA/Hex-Rays versions, database identity, and per-handler output, including site addresses and deduplicated counts. Inspect reported instructions and pseudocode. Sites are fault-oracle candidates, not conventional ROP gadgets or proof of exploitability.

A.5 Notes on Reusability

Bundled logs and the prepared database allow offline evaluation without the Pixel. New firmware versions require reviewing segment names, handler addresses, and payload registers. Function recovery is heuristic; scanner failures must not be interpreted as absence of candidates. A permanent archive, license, measured costs, and paper-specific reference results remain necessary release/evaluation metadata.

A.6 Version

Based on the LaTeX template for artifact evaluation version 1.0.